

ILLYRIAN VAULT

Technical Documentation

Version 1.5.5

Author: Orest Zogju

June 2026

A local, offline, encrypted password manager for Windows Desktop

Built with C# .NET 10 • WPF • MVVM • SQLCipher • AES-256-GCM

Table of Contents

1. Project Overview
2. Technology Stack
3. Project Structure
4. Security Architecture
5. Authentication Flow
6. Data Model & Database Schema
7. MVVM Architecture
8. UI/UX Features
9. Auto-Updater
10. Build & Deployment
11. Design Tokens

1. Project Overview

Illyrian Vault is a local, offline, encrypted password manager designed exclusively for Windows Desktop. It is built with **C# .NET 10** and **WPF** (Windows Presentation Foundation), following the **MVVM** (Model-View-ViewModel) architectural pattern. All user credentials are stored in a **SQLCipher-encrypted SQLite** database on the user's local machine. Nothing is ever sent to the cloud or transmitted over the network (except for optional breach checking via Have I Been Pwned's k-anonymity API).

The application is **Copyright 2026 Orest Zogju**. It targets **x64 Windows** and is published as a single-file **ReadyToRun** executable for optimized cold-start performance. The current version is **1.5.5**, focused on security hardening, performance optimization, and open-source release.

Key Features

- Four entry types: Login, Secure Note, Credit Card, Identity
- Two-layer encryption: SQLCipher (full-database) + AES-256-GCM (field-level)
- PBKDF2-SHA512 key derivation with 600,000 iterations
- Recovery key system for password reset without data loss
- Built-in password/passphrase generator (512-word EFF wordlist)
- Have I Been Pwned breach checking (k-anonymity)
- Three visual themes: Red (Kuq e Zi), Dark, Light
- Two languages: English and Albanian (Shqip)
- Auto-lock on idle with configurable timeout
- Clipboard auto-wipe after 15 seconds
- Brute-force defense with exponential backoff
- CSV and encrypted JSON export
- GitHub-based auto-updater

2. Technology Stack

Component	Technology	Version
Framework	.NET 10 (net10.0-windows), WPF	10.0
Architecture	MVVM via CommunityToolkit.Mvvm	8.4.0
Database	SQLite + SQLCipher AES-256	
SQLite Provider	Microsoft.Data.Sqlite.Core	9.0.5
SQLCipher Bundle	SQLitePCLRaw.bundle_e_sqlcipher	2.1.10
Icons	MahApps.Metro.IconPacks.Material	4.11.0
Language	C# (latest)	
Platform	x64 Windows only	win-x64
Build Optimizations	PublishReadyToRun + TieredPGO	

Why these choices: WPF provides the strongest MVVM ecosystem for Windows desktop apps with mature data binding and DataTemplate composition. CommunityToolkit.Mvvm eliminates boilerplate through C# source generators. SQLCipher was chosen over application-level-only encryption to provide defense in depth (full-database encryption protects schema metadata). Microsoft.Data.Sqlite.Core is used instead of Microsoft.Data.Sqlite to avoid the bundle_winsqlite3 conflict with SQLCipher.

3. Project Structure

The codebase is organized following standard MVVM conventions with clear separation of concerns:

IllyrianVault/

App.xaml / App.xaml.cs - Application entry point, global singletons, session key management

IllyrianVault.csproj - Project file with build configuration

Models/

PasswordEntry.cs - Core vault entry (Id, Title, Username, EncryptedPassword, Category...)

VaultUser.cs - User model (Username, PasswordSalt, VerificationHash, RecoveryKeyHash)

IEntryPayload.cs - Marker interface for per-category payload types

LoginPayload.cs - Login fields (Username, Password, URL, Notes)

CardPayload.cs - Card fields (CardholderName, CardNumber, Expiry, CVV)

IdentityPayload.cs - Identity fields (FullName, Email, Phone, Birthdate, Address)

NotePayload.cs - Secure note (Content)

PasswordHistory.cs - Historical password record (EntryId, EncryptedPassword)

ViewModels/

BaseViewModel.cs - Abstract base: ErrorMessage, IsBusy, HasError

AuthViewModel.cs - Orchestrates Login/Register/Recovery navigation

LoginViewModel.cs - Master password unlock + brute-force defense

RegisterViewModel.cs - 3-step vault creation wizard

RecoveryViewModel.cs - 3-step password recovery flow

MainViewModel.cs - Core vault: CRUD, search, filter, sort, clipboard, themes

AddEntryViewModel.cs - New entry creation with category selection

PasswordGeneratorViewModel.cs - Password/passphrase generator (512-word EFF list)

Views/

AuthWindow.xaml/.cs - Auth container (hosts Login/Register/Recovery)

LoginView.xaml/.cs - Login screen with profile selector

RegisterView.xaml/.cs - Registration wizard UI

RecoveryView.xaml/.cs - Password recovery wizard UI

MainWindow.xaml/.cs - Main vault: sidebar, entry list, detail panel

AddEntryWindow.xaml/.cs - Modal dialog for new entries

WindowMaximizeHelper.cs - Custom maximize for borderless windows

Services/

EncryptionService.cs - PBKDF2, AES-256-GCM, verification hash, recovery key, scoring

DatabaseService.cs - SQLite/SQLCipher DAO (profiles, schema, CRUD, history)

SecureMemory.cs - PinnedBuffer, SecureString extraction, session key mgmt

ClipboardGuard.cs - Auto-wiping clipboard with SHA-256 ownership verification

BreachCheckService.cs - HIBP k-anonymity password breach check

IdleLockService.cs - Auto-lock after configurable idle timeout

ThemeService.cs - Runtime theme switching (Red/Dark/Light)

LocalizationService.cs - Runtime language switching (EN/SQ)

ExportService.cs - CSV and encrypted JSON export

Helpers/

PayloadTemplateSelectors.cs - DataTemplateSelector for polymorphic entries

PasswordBoxHelper.cs - Attached properties for PasswordBox MVVM binding

Converters/

EqualityConverter.cs - String equality for RadioButton binding

FavoriteIconConverter.cs - Favorite star toggle

InverseBooleanConverter.cs - Boolean inversion

PasswordStrengthToBarConverter.cs - Strength score to colored bars

PasswordStrengthToBrushConverter.cs - Strength score to color brush

Resources/

Themes/ - RedTheme.xaml, DarkTheme.xaml, LightTheme.xaml

Localization/ - Strings.en.xaml, Strings.sq.xaml

Styles/ - GlobalStyles.xaml

Templates/ - PayloadTemplates.xaml

Updater/

Updater.csproj - Auto-updater project file

Program.cs - Console auto-updater via GitHub Releases API

4. Security Architecture

Security is the core design priority of Illyrian Vault. The application implements multiple layers of defense, following the principle of defense in depth.

4.1 Master Password & Key Derivation

The user's master password is never stored. Instead, it is processed through PBKDF2 (Password-Based Key Derivation Function 2) to produce a 32-byte AES-256 encryption key.

- **Algorithm:** PBKDF2-SHA256 (v1 vaults) or PBKDF2-SHA512 (v2 vaults)
- **Iterations:** 600,000 (OWASP 2023 recommended minimum)
- **Salt:** 32-byte (256-bit) generated via RandomNumberGenerator
- **Output:** 32-byte AES-256 key

Why SHA-512 for v2 vaults: GPU/ASIC mining hardware is optimized for SHA-256 (32-bit operations). SHA-512 operates on 64-bit words, which current 32-bit shader units execute less efficiently, raising the effective attacker cost by approximately 1.5-2x for the same iteration count. 600,000 iterations of SHA-512 takes approximately 400ms on a modern CPU (Core i7-1185G7), translating to roughly 2-4 million brute-force attempts per second on a high-end GPU.

4.2 Database Encryption (SQLCipher)

The entire SQLite database is encrypted using SQLCipher's AES-256 implementation. Every page of the database file is encrypted at rest, including schema metadata.

- **Key delivery:** Raw 32-byte hex key via PRAGMA key = "x'hexkey'" (bypasses SQLCipher's internal KDF)
- **cipher_page_size = 4096:** Larger pages mean fewer encrypted blocks and better I/O throughput
- **cipher_hmac_algorithm = HMAC_SHA512:** Page-level authentication using SHA-512 HMAC
- **cipher_memory_security = ON:** SQLCipher zeroes internal page cache before returning memory to allocator
- **cipher_plaintext_header_size = 0:** No unencrypted header bytes
- **WAL journal mode:** Write-Ahead Logging for better concurrent read performance

PRAGMA execution order matters: key must precede all other cipher PRAGMAs, which must precede any schema operations (CREATE TABLE). The order is: key, cipher_page_size, cipher_hmac_algorithm, cipher_kdf_algorithm, cipher_memory_security, cipher_plaintext_header_size.

4.3 Field-Level Encryption (AES-256-GCM)

Individual sensitive fields (passwords, structured payloads) are additionally encrypted with AES-256-GCM (Galois/Counter Mode), providing authenticated encryption.

- **Wire format:** [12 bytes nonce][16 bytes GCM tag][N bytes ciphertext] encoded as Base64
- **Nonce:** Fresh 96-bit (12-byte) random nonce per encryption call (NIST recommendation)
- **Tag:** 128-bit (16-byte) GCM authentication tag

GCM provides both confidentiality and integrity. Tampering with the ciphertext causes the authentication tag verification to fail during decryption, throwing an exception rather than returning corrupted plaintext.

4.4 Verification Hash

To verify the master password without opening the database, a verification hash is computed and stored in the vault.meta sidecar file.

```
verificationHash = SHA-256(derivedKey || "ILLYRIA_VERIFY")
```

The comparison uses **CryptographicOperations.FixedTimeEquals** to prevent timing-oracle attacks. Standard equality comparison short-circuits on the first differing byte; fixed-time comparison always examines ALL bytes, making execution time constant regardless of how many bytes match.

4.5 Recovery Key System

The recovery key enables password reset without losing vault data.

- **Format:** ILVT-XXXX-XXXX-XXXX-XXXX-XXXX-XXXX-XXXX (16 random bytes as 32 hex chars in 8 groups)
- **Wrapping:** Recovery key derives a wrapping key (PBKDF2-SHA512, 600k iterations, own salt). Master key is AES-GCM encrypted with wrapping key and stored in vault.meta.
- **Storage:** The recovery key itself is NEVER stored. Only a SHA-256 hash is kept in the DB. The wrapped master key + recovery salt are in vault.meta.

Recovery flow: User enters recovery key → wrapping key re-derived → wrapped master key decrypted → vault opened with old master key → new master key derived from new password → SQLCipher database re-keyed (PRAGMA rekey) → ALL individual AES-GCM fields re-encrypted with new key → vault.meta updated with new salt, hash, wrapped key.

4.6 Memory Security

The .NET garbage collector compacts the managed heap by physically moving objects, leaving stale copies of sensitive data at previous addresses. Illyrian Vault mitigates this:

- **PinnedBuffer struct:** GC-pinned byte[] (GCHandle.Alloc with Pinned type) prevents GC from moving key material
- **CryptographicOperations.ZeroMemory:** Volatile write that the JIT/optimizer cannot elide (unlike Array.Clear)
- **SecureString extraction:** Marshal.SecureStringToBSTR → pinned char[] → pinned UTF-8 byte[] (no System.String ever materialized on the managed heap)
- **Session key:** Pinned via GCHandle, zeroed on Lock/Logout/Exit via App.ClearSessionKey()
- **[MethodImpl(MethodImplOptions.NoInlining)]:** Prevents JIT tail-call optimization that could skip zeroing

4.7 Brute-Force Defense

Progressive exponential backoff prevents automated password guessing:

Attempt	1	2	3	4	5	6	7	8	9	10+
Delay (s)	0	1	2	4	8	16	30	60	120	300

- Backoff applied **BEFORE** the attempt (prevents parallel brute-force exploitation)
- Hard lockout after 10 failed attempts (300 seconds per subsequent attempt)
- Lockout state DPAPI-encrypted in vault.meta (tied to Windows user account)

4.8 Clipboard Security

- Auto-wipe after 15 seconds via CancellationToken-based scheduled task
- SHA-256 ownership hash: before wiping, verifies clipboard still contains our content
- Fixed-time hash comparison (defense in depth)
- ClearNow() called on Lock, Logout, and application exit
- Rapid-copy handling: each new copy cancels the previous wipe task

4.9 Input Validation

- Username: regex `^[A-Za-z0-9_\-]+$` with 64-char max (prevents path traversal)
- Password: minimum 8 characters, strength score $\geq 3/5$ required for registration

5. Authentication Flow

5.1 Application Startup

On startup, `App.OnStartup()` initializes `SQLCipher` (`SQLitePCL.Batteries_V2.Init()`), creates the `ThemeService` and `LocalizationService`, registers global exception handlers, and opens `AuthWindow`.

`AuthViewModel` checks if any profile directory exists under `%LOCALAPPDATA%\IllyriaVault\Profiles\`. If yes, it shows `LoginView`. If no profiles exist, it shows `RegisterView`.

5.2 Login Flow

- 1. User selects a profile (or types a username)
- 2. User enters master password (stored as `SecureString` in `PasswordBox`)
- 3. Brute-force gate: check lockout state, apply pre-attempt backoff delay
- 4. Read `vault.meta`: extract salt, stored verification hash, and version tag
- 5. Extract password from `SecureString` via `PinnedBuffer` (no managed `String`)
- 6. Derive key using `PBKDF2-SHA512` (v2) or `PBKDF2-SHA256` (v1) with 600k iterations
- 7. Compute verification hash: `SHA-256(derivedKey || "ILLYRIA_VERIFY")`
- 8. Fixed-time compare candidate hash against stored hash
- 9. On match: open `SQLCipher` database with derived key, store session key (pinned)
- 10. Fire `LoginSucceeded` event → `AuthWindow` opens `MainWindow`

5.3 Registration Flow (3 Steps)

- **Step 1:** Username (validated: alphanumeric + hyphens/underscores, max 64 chars) + master password (min 8 chars, strength $\geq 3/5$) + confirm password
- **Step 2:** Recovery key generated (`ILVT-XXXX-...`) and displayed. User **MUST** download key file before proceeding.
- **Step 3:** Confirmation summary showing username, vault path, creation date. On confirm: derive key (`SHA-512`), create profile directory, write `vault.meta` (8 lines), create `SQLCipher` DB, save user record, set session key, fire `VaultCreated` event.

5.4 Recovery Flow (3 Steps)

- **Step 1:** Enter username of the vault to recover
- **Step 2:** Enter recovery key → derive wrapping key → attempt to decrypt wrapped master key. `AES-GCM` auth failure = wrong key.
- **Step 3:** Set new password (same strength requirements) → open vault with recovered key → re-derive new master key → re-encrypt ALL `AES-GCM` fields → `PRAGMA rekey` → update `vault.meta` → set session key

5.5 Lock / Logout

Both operations: zero session key (`CryptographicOperations.ZeroMemory` + `unpin`), clear clipboard (`ClipboardGuard.ClearNow`), dispose database connection (`DisposeAsync`), fire `LockRequested` event which closes `MainWindow` and re-opens `AuthWindow`.

6. Data Model & Database Schema

6.1 Database Location

Each user gets an isolated vault at: %LOCALAPPDATA%\IllyriaVault\Profiles\{username}\vault.db

Alongside the database, a sidecar file **vault.meta** stores authentication metadata:

Line	Content	Protection
0	Salt (Base64)	Plaintext
1	Verification hash (Base64)	Plaintext
2	Username	Plaintext
3	Version tag (v1 or v2)	Plaintext
4	Consecutive failures count	DPAPI-encrypted
5	Lockout until ticks	DPAPI-encrypted
6	Recovery salt (Base64)	Plaintext
7	Wrapped master key (AES-GCM)	AES-GCM encrypted

6.2 Schema

Users Table

```
CREATE TABLE Users (Id INTEGER PRIMARY KEY AUTOINCREMENT, Username TEXT, DisplayName TEXT, PasswordSalt BLOB, VerificationHash BLOB, RecoveryKeyHash TEXT, CreatedAt TEXT)
```

PasswordEntries Table

```
CREATE TABLE PasswordEntries (Id INTEGER PRIMARY KEY AUTOINCREMENT, UserId INTEGER, Title TEXT, Username TEXT, EncryptedPassword TEXT, Url TEXT, Notes TEXT, Category TEXT DEFAULT 'Login', IsFavorite INTEGER DEFAULT 0, CreatedAt TEXT, UpdatedAt TEXT, EncryptedPayload TEXT)
```

PasswordHistory Table

```
CREATE TABLE PasswordHistory (Id INTEGER PRIMARY KEY AUTOINCREMENT, EntryId INTEGER REFERENCES PasswordEntries(Id) ON DELETE CASCADE, EncryptedPassword TEXT, CreatedAt TEXT)
```

6.3 Entry Categories

Entries are polymorphic via the **IEntryPayload** marker interface. Each category has a concrete payload class that is JSON-serialized and AES-GCM encrypted into the **EncryptedPayload** column:

- **Login:** Username, Password, URL, Notes (also stored in flat columns for search)
- **Card:** CardholderName, CardNumber, Expiry, CVV
- **Identity:** FullName, Email, Phone, Birthdate, Address
- **Note:** Content (free-form text)

7. MVVM Architecture

7.1 CommunityToolkit.Mvvm Source Generators

The project uses CommunityToolkit.Mvvm's source generators to eliminate MVVM boilerplate. All ViewModel classes are marked **partial** so the generator can create companion partial classes.

- **[ObservableProperty]**: Generates a public property with INotifyPropertyChanged from a private field
- **[NotifyPropertyChangedFor]**: Fires additional PropertyChanged events when the decorated property changes
- **[NotifyCanExecuteChangedFor]**: Re-evaluates command CanExecute when the property changes
- **[RelayCommand]**: Generates an ICommand property from a method, with optional CanExecute support

7.2 Navigation Pattern

The app uses **ViewModel-first navigation**. AuthViewModel has a CurrentViewModel property of type object?. When it assigns a LoginViewModel, RegisterViewModel, or RecoveryViewModel to CurrentViewModel, WPF's DataTemplate system in App.xaml automatically resolves and displays the correct View.

7.3 BaseViewModel

All ViewModels inherit from BaseViewModel, which provides: **ErrorMessage** (string, clears via ClearError()), **IsBusy** (bool, gates async commands), and **HasError** (computed from ErrorMessage). This ensures consistent error display and loading state across all screens.

7.4 PasswordBox MVVM Bridge

WPF's PasswordBox doesn't expose a bindable Password property (by design, for security). PasswordBoxHelper provides two attached property modes:

- **Secure mode**: Binds PasswordBox.SecurePassword to a SecureString ViewModel property. Password never becomes a managed string.
- **Legacy string mode**: Binds PasswordBox.Password to a string property. Used only for RegisterViewModel's live strength-score animation (requires character access).

Each mode uses a per-instance DependencyProperty guard (not a static bool) to prevent re-entry loops during bi-directional updates.

7.5 Polymorphic Entry Rendering

DataTemplateSelector classes (PayloadViewSelector, PayloadEditSelector) inspect the IEntryPayload concrete type and return the appropriate DataTemplate. This enables a single detail panel to render Login, Card, Identity, and Note entries with completely different layouts.

8. UI/UX Features

8.1 Themes

- **Red (Kuq e Zi):** Obsidian #0B0B0C background + blood-red #D61C22 accent (default)
- **Dark:** Slate/navy #16161A background + indigo #6366F1 accent
- **Light:** Light gray #F5F6F8 background + deep indigo #4F46E5 accent

Themes are implemented as ResourceDictionary XAML files with consistent brush keys (BrushWindow, BrushSidebar, BrushCard, BrushAccent, BrushText, etc.). ThemeService pre-parses all three dictionaries at startup and switches via reference swap (Remove old + Add new) for instant theme changes.

8.2 Localization

- **English (EN):** Default language
- **Albanian (Shqip, SQ):** Full translation

Strings are stored as DynamicResource entries in Strings.en.xaml and Strings.sq.xaml. LocalizationService swaps the active dictionary at runtime. The indexer (localization["key"]) provides ViewModel-level access.

8.3 Main Window Layout

- Sidebar navigation: All Items, Favorites, Logins, Notes, Cards, Identities, Generator, Settings
- Badge counts per category (live-updated on add/delete/favorite)
- Entry list with search, category filter, and sort (Recently Used, A-Z, Z-A, Date Created)
- Detail panel with view/edit modes, password reveal, copy actions
- Custom borderless window with AllowsTransparency for rounded corners on Win10

8.4 Password Generator

- **Character mode:** Configurable length, uppercase, lowercase, numbers, symbols
- **Passphrase mode:** 2-10 words from a 512-word EFF-derived wordlist, hyphen-separated
- Uses RandomNumberGenerator.GetInt32 for cryptographic randomness
- Available both standalone (Settings > Generator) and inline (per-entry edit mode)

8.5 Breach Check

Integrated Have I Been Pwned check using the k-anonymity range API. Only the first 5 characters of the SHA-1 hash are sent. Visual status indicators: Not Checked, Checking, Safe, Breached, Error.

8.6 Additional Features

- Password history: auto-archived on change, viewable per-entry with reveal/hide
- Toast notifications for copy and save actions
- Configurable idle auto-lock: 1, 5, 10, 15, 30 minutes
- Export: CSV (plaintext) and encrypted JSON
- Favorite entries with quick-access sidebar section

9. Auto-Updater

A separate console application (Updater.csproj, .NET 10) handles automatic updates via the GitHub Releases API.

Update Flow

- 1. Read the FileVersion from the currently installed IllyrianVault.exe
- 2. Query GitHub API for the latest release tag
- 3. Compare versions; exit if already up to date
- 4. Prompt user for confirmation (showing version change and download size)
- 5. Close any running instances of Illyrian Vault (CloseMainWindow + WaitForExit)
- 6. Download the new .exe with a visual progress bar
- 7. Swap files: rename old exe (Windows allows renaming running exes), drop in new exe, delete old
- 8. Launch the updated application

The updater handles asset name discrepancies (GitHub converts spaces to dots in filenames) and falls back to the first .exe asset if the expected name is not found.

10. Build & Deployment

Build Configuration

- **PublishReadyToRun = true:** Pre-JITs all IL to native code, eliminating cold-JIT cost on startup
- **TieredPGO = true:** Profile-guided optimization re-compiles hot paths at runtime
- **Platform:** x64 only (Platforms=x64, RuntimeIdentifier=win-x64)
- **Output:** WinExe (no console window)

Publish Command

```
dotnet publish -c Release -r win-x64 --self-contained
```

Repository

GitHub: [Orest-Z/Illyria-Vault](#)

11. Design Tokens

Design tokens derived from the HTML/CSS mockup reference:

Typography

- **Primary font:** Segoe UI Variable Text (Windows system font)
- **Monospace font:** JetBrains Mono / Cascadia Code

Border Radii

Usage	Radius
Chip	4px
Small button	6px
Input / Nav item / Button	8px
Chip card	10px
Card	12px
Detail tile	14px
Hero panel	16px

Window Sizes

Window	Width	Height
Login (AuthWindow)	980px	640px
Register (AuthWindow)	820px	700px
Main Window	1320px	800px

End of Document - Illyrian Vault Technical Documentation v1.5.5